

- 学号: 221240093
- 姓名: 陈力峥
- 邮箱: 221240093@smail.nju.edu.cn

编译

编译采用 `Makefile` , 在 `Code` 目录下执行 `make` 即可, 随后生成 `parser` 可执行文件。

`Code` 目录下运行 `parser`: `./parser [input_file] [output_file]`

实验内容

完成实验三所有部分:

- C-- 语言的中间代码到MIPS汇编的转换
 - 包含指令选择、寄存器分配、栈帧管理等部分

实验细节

1. 主要实现文件

- `assembly.c/h` : 实现中间代码到MIPS汇编的转换
- `addr_regs.c/h` : 实现寄存器分配和地址描述符管理
- 对于实验三公开出来的测试用例对实验四进行了测试, 由于假设不包含结构体, 只对 `A1~A5 B1~B3 C1~C2` 进行了测试并通过测试。

2. 目标代码生成实现细节

- 目前的栈帧分配策略如下:
 - a. 函数活动记录

栈帧布局（从高地址到低地址）：

```

+-----+
| 参数区域          |
+-----+
| 返回地址($ra)      |
+-----+
| 旧帧指针($fp)      |
+-----+
| 保存的寄存器        |
+-----+
| 局部变量            |
+-----+

```

b. 寄存器分配

- 采用朴素寄存器分配算法
- 使用18个通用寄存器（*t0*—*t9*, *s0*—*s7*）（地位均等）
- 维护寄存器描述符和地址描述符（在Allocate分配以及 *lw sw* 时需要维护这两个结构体）
- 当寄存器不足时，将变量溢出到内存

c. 地址描述符管理

- 使用符号表记录变量的栈偏移量
- 跟踪变量是否在寄存器中
- 维护变量的内存状态

• 关键实现思想

- 栈帧管理：使用 *fp* 作为帧指针，相对于 *fp* 计算变量偏移，引入中间代码的符号表，当插入局部变量的时候，同时也维护移动 *sp*
- 寄存器分配：优先使用空闲寄存器，必要时溢出到内存(如计算完成的结果直接进行 *sw*)
- 变量访问：优先从寄存器读取，不在寄存器则从内存加载(通过符号表得到地址描述符，根据地址描述符维护的信息来判断)
- 函数调用：保存返回地址和帧指针，处理参数传递(读取参数的顺序 ARG PARAM 分别特殊处理)
- 比较值得一提的是对于数组的处理，数组的声明是 DEC 语句，例如数组 `int arr[4]`，此时进行如下处理：
 - 将名为 "&arr" 的变量插入符号表，大小为 4 字节，随后将不需要名字的变量插入符号表代表数组的数据部分，大小为 16 字节，此时对于数据的 *sw lw* 操作手动加上了头部的 4 字节偏移量，这样可以比较方便的存储地址信息以及数组数据。